App Development Project Report

App Dev Project Report

1. Student Details

- **Name**: Dnyaneshwari Mahajan
- **Roll Number**: 21f3001176
- **Email:** 21f3001176@ds.study.iitm.ac.in
- **About Me**: I am student of IIT Madras BS Degree program with a strong interest in web application development and data-driven technologies. I enjoy building meaningful applications that combine learning, analytics, and user experience.

2. Introduction :

Project Title : Hospital Management

Problem Statement : To develop a web-based system that simplifies doctor–patient interactions by enabling patients to search doctors, view availability, and book appointments, while allowing doctors to manage schedules efficiently

Approach : The system was developed using **Flask** as the backend framework, combined with **SQLite** for lightweight data storage, and **Bootstrap + Jinja** for a responsive frontend. A modular project structure was followed to separate routes, models, templates, and static files.Patients can browse doctor profiles, view real-time availability, and book appointments, while doctors can manage their calendars and update their slots. Dynamic rendering using Jinja templates ensures smooth user interaction, and Bootstrap makes the UI clean and accessible across devices.

3. AI/LLM Declaration:

I used ChatGPT (GPT-5) to assist in creating API documentation samples, and improving variable naming consistency. The extent of AI/LLM usage is around 10–15%, limited to code suggestions and documentation formatting

4. Technologies and Frameworks Used :

Technologies Used:

- **Backend:** Flask (Python)
- **Database:** SQLite
- **Frontend:** HTML, Bootstrap, Jinja2 Templates
- **ORM:** SQLAlchemy

System Components:

- **User Authentication:** Secure login system with role-based access for doctors and patients.
- **Doctor Availability Management:** Doctors can add, update, and manage their availability slots.
- **Patient Appointment Booking:** Patients can search doctors by specialization, view available slots, and book appointments.
- **Appointment Tracking:** Stores appointment details, booking history, and future schedules for both doctors and patients.

5. Database Schema / ER Diagram

Tables:

- **Users** — stores user information and doctor details
  (user_id, username, email_id, password, role, contact_no, created_at, blacklist, department_id, qualifications, specialty, years_experience, years_specialist, bio, photo_url)
- **DoctorAvailability** — stores each doctor's available dates and slot timings
  (id, doctor_id, date, slot_1, slot_2)
- **Appointment** — stores patient appointments with doctors
  (id, status, doctor_id, date, time, user_id, treatment_id)
- **Departments** — stores hospital department details
  (id, dept_name, description)
- **Treatment** — stores treatment records linked to appointments
  (id, diagnosis, treatment, prescription)

Relationships:

- **One-to-Many** → Department 1 → Users(doctor)
- **Many-to-One** → User (doctor) 1 → Appointments
- **Many-to-One** → User (patient) 1 → Appointments
- **Many-to-One** → User (doctor) 1 → DoctorAvailability
- **Appointment 1** → 1 Treatment

Entity-Relationship (ER) Diagram

6.API Resource Endpoints

Auth Routes:

Endpoint	Method	Description
/	GET	Home (Landing) page → *index.html*
/auth/register	GET/POST	User registration page → *register.html*
/auth/login	GET/POST	User login page → *login.html*

Admin Routes :

Endpoint	Method	Description
/admin/dashboard	GET	Admin Dashboard → admin_dash.html

Endpoint	Method	Description
/admin/create_doctor	GET/POST	Create doctor with details → create_doctor.html
/admin/delete_user/<int:user_id>	POST/GET	Delete user from database
/admin/edit_patient/<int:user_id>	GET/POST	Edit patient details → edit_patient.html
/admin/edit_doctor/<int:user_id>	GET/POST	Edit doctor details → edit_doctor.html
/admin/blacklist/<int:user_id>	POST/GET	Blacklist any user (Redirects to admin dashboard)
/admin/history	GET	View full history of all patients & doctors → history.html

Doctor Routes :

Endpoint	Method	Description
/doctor/dashboard	GET	Doctor Dashboard → doc_dash.html
/doctor/history	GET	History of all patients treated by doctor → history.html
/doctor/availability	GET/POST	Create/Update doctor availability slots → availability.html
/doctor/appointment/<int:appt_id>/update	GET/POST	Manage appointment (CRUD) → Redirect to dashboard
/doctor/complete_appointment/<int:appt_id>	POST/GET	Mark appointment as completed

Patient Routes

Endpoint	Method	Description
/patient/dashboard	GET	Patient Dashboard → *patient_dash.html*
/patient/history	GET	View full appointment history → *history.html*
/patient/booking_chart/<int:doctor_id>	GET	View doctor's available slots → *slot_booking_chart.html*
/patient/book_slot	POST	Book appointment slot (CRUD)
/patient/cancel_appointment/<int:appt_id>	POST/GET	Cancel appointment (Update)

Endpoint	Method	Description
`/admin/edit_patient/<int:user_id>`	GET/POST	Edit patient details → *edit_patient.html*

7.Architecture and Features:

Architecture Overview:

- **app.py** – main Flask application entry point
- **/models** – database models using SQLAlchemy
- **/routes** – Flask Blueprints for user and activity routes
- **/templates** – Jinja2 HTML templates
- **/static** – CSS, JS, and chart visualization files

Implemented Feature:

User Roles:

- **Admin:**
  - Can create and manage doctor accounts.
  - Can edit doctor details and blacklist/unblacklist users and doctors.
  - Can view complete appointment and treatment history across the system.
- **Patient:**
  - Can search doctors by specialization.
  - Can view their own appointment history.
  - Can book appointments based on available time slots.
  - Can cancel their booked appointments.
- **Doctor:**
  - Can view all their patients' appointment and treatment history.
  - Can create, update, and manage availability time slots.
  - Can record diagnosis, treatment, and prescriptions.
  - Can mark appointments as completed

VIDEO LINK:

Video Link
https://drive.google.com/file/d/1j2DRQzEjd67FKamthr7qFl1Nir07lmSo/view?usp=drive_link